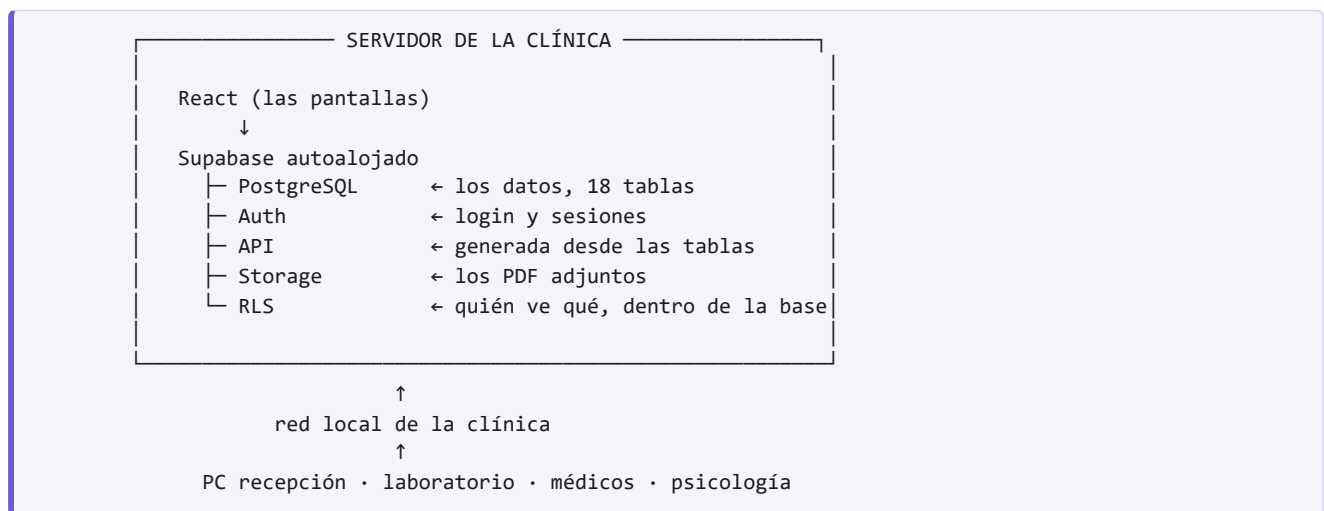


Estructura del proyecto

Cómo se organiza el código — CML NOA · Incremento 1: Prelaboral

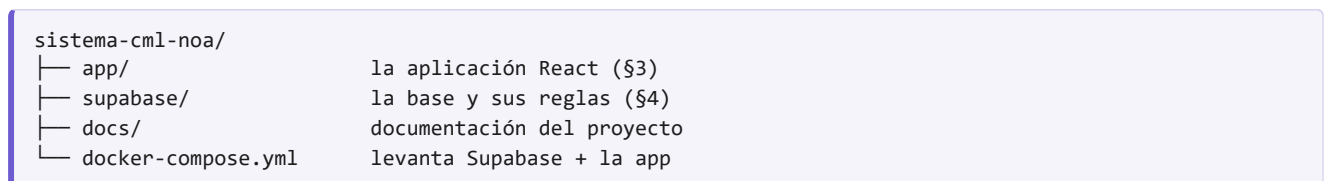
Uso interno de Escencial Consultora Fecha: Septiembre de 2026

1 · El stack, en una imagen



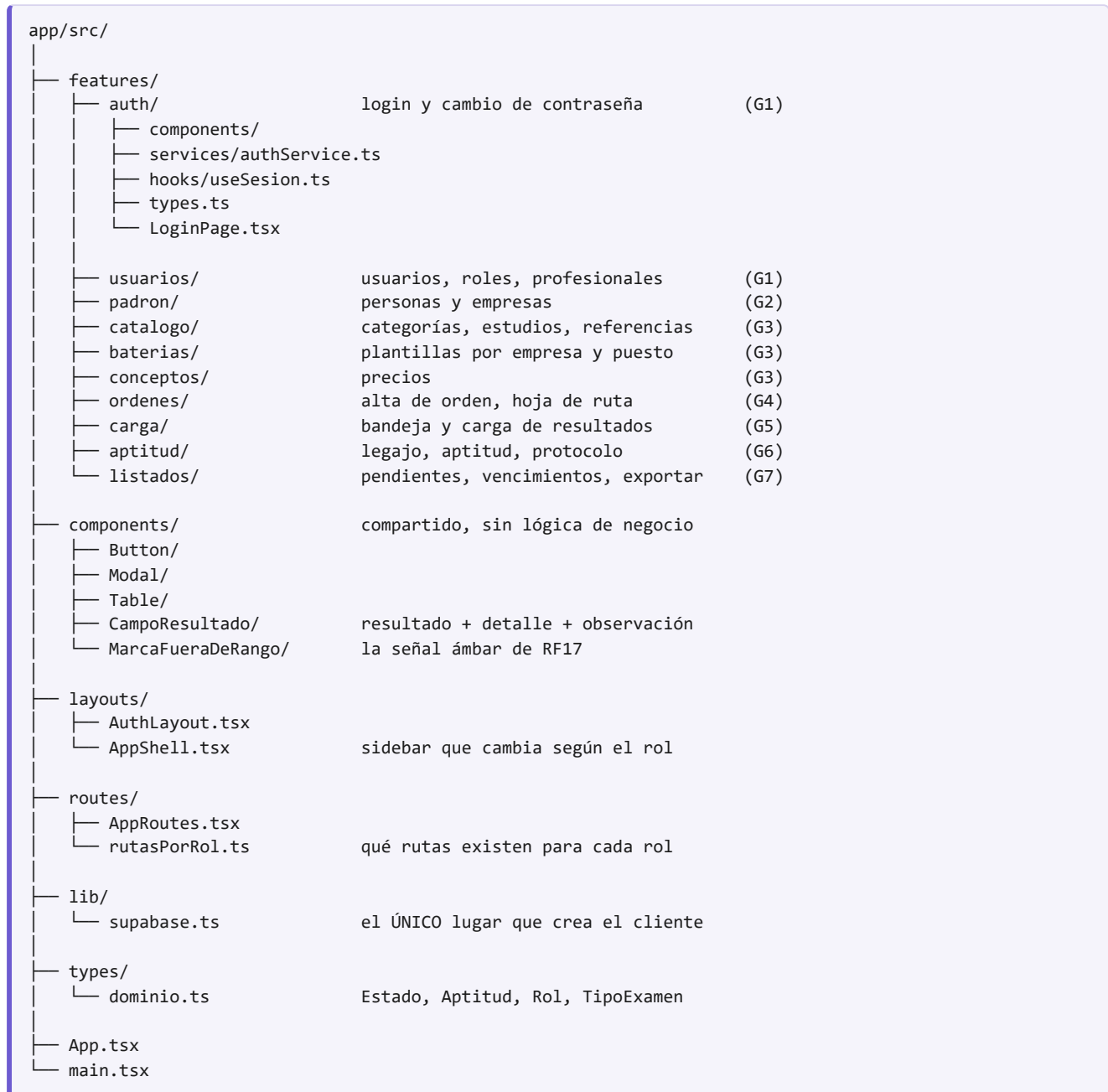
Todo corre adentro del edificio. Supabase autoalojado es el mismo software del servicio en la nube, instalado en la PC del consultorio: es código abierto, no valida licencia contra internet y funciona con el cable de red desenchufado del router.

2 · El repositorio



3 · La aplicación — `app/src/`

Organizada **por funcionalidad**, no por tipo de archivo. Cada carpeta de `features/` es autosuficiente: sus pantallas, sus componentes, sus llamadas a la base y sus tipos viven juntos.



Las diez carpetas de `features/` son los siete grupos de la SRS más `auth`. Buscar dónde está algo no requiere adivinar: si es de baterías, está en `features/baterias/`.

Dos reglas de esta estructura

`lib/supabase.ts` es el único que crea el cliente. Todos los servicios lo importan de ahí. Así la sesión, el manejo de errores y el "sesión vencida" se resuelven en un lugar, no en diez.

`types/dominio.ts` fija el vocabulario. Es lo que evita que la pantalla diga una cosa y la base otra:

```

export type EstadoOrden = "ABIERTA" | "EN_CURSO" | "COMPLETA" | "INFORMADA"
export type Aptitud      = "PENDIENTE" | "APTO" | "NO_APTO"
export type TipoExamen   = "PRELABORAL" | "PERIODICO" | "EGRESO"
export type Rol           = "R1" | "R2" | "R3" | "R4" | "R5" | "R6" | "R7" | "R8"
    
```

Sin esto ya apareció el problema: la maqueta usaba `completo` donde el esquema dice `COMPLETA`. Con un tipo, eso no compila. Es el mismo error que en Carnicerías costó caro (`cajero` contra `vendedor`), y ahí quedó escrito:

"mientras no coincidan, ninguna política de ese rol se aplica".

4 · La base — [supabase/](#)

```
supabase/
├── migrations/
│   ├── 001_esquema.sql          las 18 tablas y las 3 vistas
│   ├── 002_catalogo.sql        9 categorías, 122 estudios, 33 conceptos
│   ├── 003_datos_base.sql      roles, profesionales, empresas, baterías
│   └── 004_politicas_rls.sql    quién ve qué ← la seguridad real
├── functions/
│   ├── crear_orden.sql          copia la batería según el sexo, numera, congela el importe
│   ├── calcular_presupuesto.sql recorre los conceptos sin duplicar estudios
│   └── marcar_fuera_de_rango.sql compara contra ref_h o ref_m según la persona
└── seed/
    └── usuarios_demo.sql        solo para el entorno de prueba, nunca en producción
```

Por qué las tres funciones viven en la base y no en la pantalla

Las tres son reglas que **no pueden depender de qué pantalla las llame**:

Función	Qué pasaría si viviera en el frontend
crear_orden	Dos puestos creando una orden al mismo tiempo podrían sacar el mismo número
calcular_presupuesto	Cada pantalla podría calcular distinto, y el importe congelado quedaría mal
marcar_fuera_de_rango	Un resultado cargado desde otro lado entraría sin evaluar

5 · Dónde vive la seguridad

No en las carpetas. Organizar por features hace el código mantenible; no protege un solo dato.

Con Supabase, la aplicación del navegador **habla con la base usando una clave pública** que cualquiera puede leer. Esconder un botón no impide nada.

La defensa es 004_politicas_rls.sql: reglas dentro de la base que se evalúan en cada consulta, venga del sistema o de alguien armando el pedido a mano.

```
-- El bioquímico ve solo los estudios de sus categorías
create policy lab_ve_lo_suyo on orden_estudio for select
using (
  exists (
    select 1 from estudio e
    join categoria c on c.id = e.categoria_id
    where e.id = orden_estudio.estudio_id
    and c.rol_carga = mi_rol()
  )
);

-- La aptitud la fija solo el médico laboral – ni el administrador
create policy solo_medico_laboral_fija_aptitud on orden for update
using (mi_rol() = 'R3');
```

Tres reglas que no se negocian:

1. **El alcance sale de la sesión, nunca del pedido.** Nada de `?empresa_id=15`: si el cliente elige el filtro, cambia el número y ve otra empresa. 2. **Ninguna pantalla decide permisos.** Si mañana alguien borra una validación del frontend, la base sigue diciendo que no. 3. **Toda tabla nace con RLS activo.** Una tabla nueva sin política no devuelve nada — falla cerrada, no abierta.

6 · Cómo se despliega

```
docker-compose.yml
├─ supabase (varios contenedores)  ← base, auth, API, storage
└─ app                             ← las pantallas, servidas por nginx
```

Lo que no se copia de la maqueta: el `deploy.sh` que hace `git reset --hard` automático desde un repositorio público, y el túnel que publica la aplicación en internet. Para un prototipo descartable está bien; **en el servidor de la clínica, ninguna de las dos cosas entra** — contradicen que el servidor no sea alcanzable desde afuera (RNF-06, RNF-09) y que corra en una PC dedicada (RNF-30).

En producción el despliegue es manual y deliberado: se prueba, se decide, se actualiza.

7 · Qué se hereda de la maqueta y qué se descarta

Se conserva	Se descarta
El <code>DESIGN.md</code> : colores, tipografía y las reglas de acento y semáforo	<code>src/mock/</code> entero — lo reemplaza la base real
La navegación por rol desde config, con un solo sidebar	Los usuarios y contraseñas de prueba
La capa de servicios como frontera única hacia los datos	El <code>deploy.sh</code> y el túnel
Las pantallas ya validadas de login y bandeja	Los estados y roles desactualizados: hay que emparejarlos con <code>types/dominio.ts</code>